

HDOC Day-23 — Basic Square/Rectangular Patterns Using Nested Loops in C

Name- Aaryan Pratap Singh

Sap Id - 590032842

Below is a complete solution with the **algorithm**, **test-case table**, **C program**, and **sample execution**. The program uses a **switch statement**, **nested loops**, **no arrays**, and **separate parameterless void functions** for all four patterns.

1. Algorithm

Main Program

1. Start.
2. Declare **choice** as an integer.
3. Display the menu:
 - Solid square pattern
 - Solid rectangular pattern
 - Row-number pattern
 - Column-number pattern
 - Exit
4. Read **choice**.
5. Use **switch(choice)**:
 - If **choice** = 1, call **solidSquare()**.
 - If **choice** = 2, call **solidRectangle()**.
 - If **choice** = 3, call **rowNumberPattern()**.
 - If **choice** = 4, call **columnNumberPattern()**.
 - If **choice** = 5, exit the program.
 - Otherwise, display an invalid-choice message.
6. Repeat the menu until the user selects **5**.
7. Stop.

Function 1: **solidSquare()**

1. Input **n**.
2. Check whether **n** is positive.
3. Use an outer loop from 1 to **n** for the rows.
4. For each row, use an inner loop from 1 to **n**.
5. Print ***** in the inner loop.
6. Move to the next line after each row.
7. Return to **main()**.

Function 2: **solidRectangle()**

1. Input the number of rows m and columns n .
2. Check whether both values are positive.
3. Use an outer loop from 1 to m .
4. Use an inner loop from 1 to n .
5. Print $*$ in the inner loop.
6. Move to the next line after every row.
7. Return to `main()`.

Function 3: `rowNumberPattern()`

1. Input m and n .
2. Check whether both are positive.
3. Use an outer loop i from 1 to m .
4. Use an inner loop j from 1 to n .
5. Print the value of i in every column of that row.
6. Move to the next line.
7. Return to `main()`.

Function 4: `columnNumberPattern()`

1. Input m and n .
2. Check whether both are positive.
3. Use an outer loop i from 1 to m .
4. Use an inner loop j from 1 to n .
5. Print the value of j .
6. Move to the next line after each row.
7. Return to `main()`.

2. Test-Case Table

Test Case	Choice	Input	Expected Output	Actual Output	Result
1	1	$n = 4$	4×4 solid square	4×4 solid square	Pass
2	2	$m = 3, n =$	3×5 solid rectangle	3×5 solid rectangle	Pass
3	3	$m = 4, n = 4$	Rows contain 1, 2, 3, 4 respectively	Correct row-number pattern	Pass
4	4	$m = 4, n =$	Each row contains 1 2 3 4	Correct column-number	Pass
5	1	$n = 1$	*	*	Pass
6	2	$m = 0, n =$	Error message	Error message displayed	Pass
7	3	$m = 3, n = -2$	Error message	Error message displayed	Pass
8	—	Choice 7	Invalid-choice message	Invalid-choice message displayed	Pass
9	5	—	Program terminates	Program terminates	Pass

3. C Program

```
#include <stdio.h>

/* Function declarations */

void solidSquare(void);
void solidRectangle(void);
void rowNumberPattern(void);
void columnNumberPattern(void);

int main(void)
{
    int choice;

    do
    {
        printf("\n==== PATTERN MENU =====\n");
        printf("1. Solid square pattern\n");
        printf("2. Solid rectangular pattern\n");
        printf("3. Row-number pattern\n");
        printf("4. Column-number pattern\n");
        printf("5. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                solidSquare();
                break;

            case 2:
                solidRectangle();
                break;

            case 3:
                rowNumberPattern();
                break;

            case 4:
                columnNumberPattern();
                break;
        }
    }
}
```

```

        case 5:
            printf("Exiting program.\n");
            break;

        default:
            printf("Invalid choice. Please enter a choice
from 1 to 5.\n");
    }

    } while (choice != 5);

    return 0;
}

/* Solid square pattern */
void solidSquare(void)
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n; j++)
        {
            printf("* ");
        }
        printf("\n");
    }
}

/* Solid rectangular pattern */
void solidRectangle(void)
{
    int m, n, i, j;

    printf("Enter number of rows and columns: ");
    scanf("%d%d", &m, &n);

```

```

    if (m <= 0 || n <= 0)
    {
        printf("Rows and columns must be positive.\n");
        return;
    }

    for (i = 1; i <= m; i++)
    {
        for (j = 1; j <= n; j++)
        {
            printf("* ");
        }
        printf("\n");
    }
}

/* Row-number pattern */
void rowNumberPattern(void)
{
    int m, n, i, j;

    printf("Enter number of rows and columns: ");
    scanf("%d%d", &m, &n);

    if (m <= 0 || n <= 0)
    {
        printf("Rows and columns must be positive.\n");
        return;
    }

    for (i = 1; i <= m; i++)
    {
        for (j = 1; j <= n; j++)
        {
            printf("%d ", i);
        }
        printf("\n");
    }
}

/* Column-number pattern */
void columnNumberPattern(void)
{
    int m, n, i, j;

```

```

printf("Enter number of rows and columns: ");
scanf("%d%d", &m, &n);

if (m <= 0 || n <= 0)
{
    printf("Rows and columns must be positive.\n");
    return;
}

for (i = 1; i <= m; i++)
{
    for (j = 1; j <= n; j++)
    {
        printf("%d ", j);
    }
    printf("\n");
}
}

```

5. Execution

Choice 1 — Solid Square

```

[aaryanrajput@aaryans-macbook ~ % nano daay23.c
[aaryanrajput@aaryans-macbook ~ % clang daay23.c -o daay23.c
[aaryanrajput@aaryans-macbook ~ % ./daay23.c
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 1
Enter number of rows: 4
* * * *
* * * *
* * * *
* * * *
aaryanrajput@aaryans-macbook ~ % █

```

Choice 2 — Solid Rectangle

```
[aaryanrajput@aaryans-macbook ~ % ./daay23.c
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 2
Enter number of rows: 3
Enter number of columns: 4
* * * *
* * * *
* * * *
aaryanrajput@aaryans-macbook ~ %
```

Choice 3 — Row-Number Pattern

```
[aaryanrajput@aaryans-macbook ~ % ./daay23.c
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 3
Enter number of rows: 4
Enter number of columns: 4
1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4
aaryanrajput@aaryans-macbook ~ %
```

Choice 4 — Column-Number Pattern

```
[aaryanrajput@aaryans-macbook ~ % ./daay23.c
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 4
Enter number of rows: 4
Enter number of columns: 4
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4
aaryanrajput@aaryans-macbook ~ %
```

The program was **compiled and executed successfully**, and the four required patterns produced the expected results.